

```
shubham@k8s-minikube:~$ minikube kubectl -- get nodes -o wide
NAME      STATUS    ROLES    AGE      VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUNTIME
minikube   Ready     control-plane,master   5m42s    v1.23.3   192.168.49.2   <none>        Ubuntu 20.04.2 LTS   5.4.0-109-generic   docker://20.10.12
shubham@k8s-minikube:~$
```

Figure 4: Kubernetes node in minikube cluster with default container runtime

```
shubham@k8s-minikube:~$ minikube kubectl -- get pods --all-namespaces -o wide
NAMESPACE   NAME                                     READY   STATUS    RESTARTS   AGE   IP            NODE      NOMINATED NODE   READINESS GATES
kube-system  coredns-64897985d-4mkd9                1/1     Running   0           4m58s  172.17.0.2    minikube  <none>           <none>
kube-system  etcd-minikube                           1/1     Running   0           5m19s  192.168.49.2  minikube  <none>           <none>
kube-system  kube-apiserver-minikube                 1/1     Running   0           5m19s  192.168.49.2  minikube  <none>           <none>
kube-system  kube-controller-manager-minikube        1/1     Running   0           5m13s  192.168.49.2  minikube  <none>           <none>
kube-system  kube-proxy-zjbt1                        1/1     Running   0           4m58s  192.168.49.2  minikube  <none>           <none>
kube-system  kube-scheduler-minikube                 1/1     Running   0           5m19s  192.168.49.2  minikube  <none>           <none>
kube-system  storage-provisioner                     1/1     Running   1 (4m22s ago)  5m6s   192.168.49.2  minikube  <none>           <none>
shubham@k8s-minikube:~$
```

Figure 5: All the pods in Kubernetes minikube

started and stopped like any other Linux service. minikube can run components of Kubernetes such as pod, Ingress, service, and others. For minikube to be deployed the machine must have either container or virtual machine managers such as Docker, Hyperkit, HyperV, Kernel-based Virtual Machine (KVM), Parallels, Podman, VirtualBox, or VMware Fusion/Workstation installed on the host machine.

The local installation was done on an Ubuntu 20.04.2 LTS virtual machine, with two CPUs, 8GB memory, and 50GB storage.

Next, we updated and upgraded the target platform:

```
$ sudo apt update
$ sudo apt upgrade
```

The following commands get downloaded and install the latest minikube stable release on the target platform:

```
$ curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
$ sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

And that's it; Kubernetes is successfully installed over the node. To start the Kubernetes cluster, run the following command:

```
$ minikube start
```

Figure 4 shows all the nodes

attached to the Kubernetes cluster and proves the point that minikube is a single-node Kubernetes cluster that acts as a control plane (master) node under ROLES. The STATUS tells whether the node is active or inactive. The AGE tells the user how long the node has been active. VERSION depicts the version of the minikube installation.

The minikube cluster is connected to a network, and the network settings give an IP address to the node, which is read as INTERNAL-IP. No EXTERNAL-IP is attached to the node; that's why we see none here. As mentioned earlier, the installation is done over an Ubuntu-based system, and the OS-IMAGE showcases the same. In KERNEL-VERSION we see the version of the kernel of the OS platform. Finally, minikube uses the container runtime to achieve the life cycle of the containers within the cluster, and CONTAINER-RUNTIME shows which version of container runtime it uses internally as well as its version.

Figure 5 shows all the pods attached to all the name spaces in the minikube cluster. Under NAMESPACES, all the name spaces attached to the minikube cluster are visible. NAME showcases the names of all the pods that are running. READY demonstrates the number of pods running as against the number of pods required. All the pods are running, which is evident

from STATUS. If there is any restart in any particular pod, that can be seen under RESTARTS. minikube creates an internal subnet, and some pods are connected to it, while other pods are connected to the internal IP address of the cluster. NODE shows which place each pod is running at. Since we went with the default settings, there was no priority set between nodes -- that's why NOMINATED NODE shows none. READINESS GATES helps implement custom checks for any running pod. Since we went with default settings, it shows none.

Setting up a Kubernetes cluster using KVM

A Kubernetes cluster consists of more than a single node (minikube). The best practice is to have multiple worker nodes to enable high availability. This Kubernetes cluster is managed by the user via the kubeadm tool. Container runtime is deployed over each node of the cluster so that it manages the life cycle of the pods. We'll learn how to facilitate a Kubernetes cluster over multiple KVM nodes (master and workers) with common container runtimes such as:

- CRI-O
- Docker Engine
- Containerd

The setup of the Kubernetes cluster was done on two Ubuntu 20.04.4 LTS virtual machines. Here the master and worker node configurations were two CPUs, 4GB of memory, and 50GB of storage.